Masterclass: *AI/ML Benchmarking*

How to turn a model comparison into a scientific benchmark that anyone can rerun, score and compete on

arXiv:2511.05614 mlcommons-science.github.io/benchmark

Ben Hawks · Gregor von Laszewski · Matthew D. Sinclair · Marco Colombo · Shivaram Venkataraman · Rutwik Jain · Yiwei Jiang · Nhan Tran · Geoffrey Fox
Fermilab · University of Virginia · University of Wisconsin–Madison · University of Illinois Urbana-Champaign

Ben Hawks · Fermilab — *CMS masterclass*
26 October 2026



74 benchmark tasks by AI/ML motif and domain — HEP is the busiest column

The hour — *four parts*

Part I **Why benchmark** — what a benchmark is for, and why so few “benchmarks” can be rerun

Part II **What makes a good benchmark** — five elements, a rubric to score them, and the bar for “high quality”

Part III **Where HEP stands** — 74 scored science benchmarks: HEP brings the most, and few clear the bar

Part IV **Benchmarking well** — the question, baselines, metrics, splits and leakage, error bars, reading a result, score it yourself

Part I — *why benchmark*

Part I **Why benchmark** — what a benchmark is for, and why so few “benchmarks” can be rerun

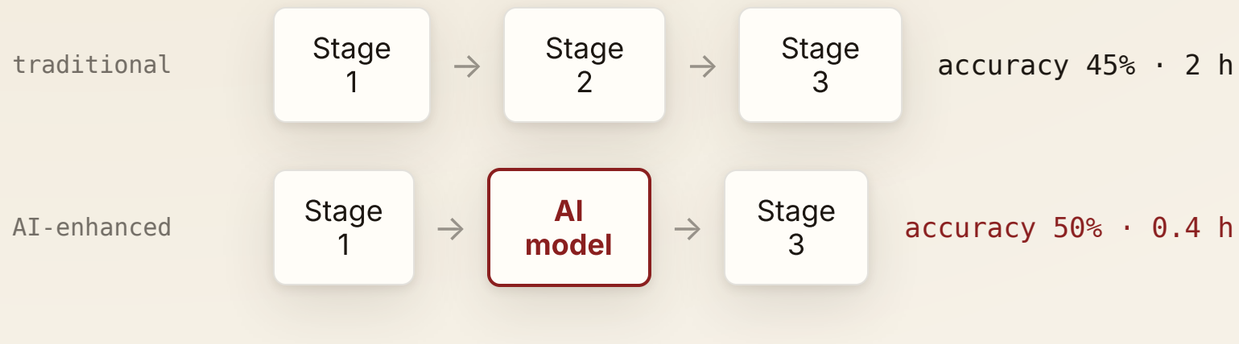
Part II **What makes a good benchmark** — five elements, a rubric to score them, and the bar for “high quality”

Part III **Where HEP stands** — 74 scored science benchmarks: HEP brings the most, and few clear the bar

Part IV **Benchmarking well** — the question, baselines, metrics, splits and leakage, error bars, reading a result, score it yourself

Concrete evaluation with *meaningful* metrics

- **Guide development**
  which AI approach actually works best, not which one looked best in one run
- **Prove AI advantage**
  a fair, apples-to-apples comparison against the traditional workflow
- **Both need a benchmark: a fixed task, fixed data, a fixed scorer**
  so “better” means the same thing to everyone who runs it



illustrative numbers — your benchmark's metrics decide what “better” means

in CMS terms

- swap a reconstruction, simulation or trigger stage for a model
- is it better? on which metric, under which latency or resource budget, against which baseline?

TERMS **AI advantage** a measured gain over the traditional workflow, on the benchmark’s own metrics

THE STATE OF SCIENTIFIC ML BENCHMARKING

Plenty of benchmarks. Few you can *actually rerun*.

“Often, a dataset is provided without concrete tasks, the code is not reproducible, or the metrics are undefined.”

An MLCommons Scientific Benchmarks Ontology, §I

no task · no reproducible code · no defined metric · siloed per domain

TERMS **MLCommons** the open engineering consortium behind the MLPerf benchmarks **ontology** a structured catalogue: shared definitions, categories and ratings

Part II — *what makes a good benchmark*

Part I **Why benchmark** — what a benchmark is for, and why so few “benchmarks” can be rerun

Part II **What makes a good benchmark** — five elements, a rubric to score them, and the bar for “high quality”

Part III **Where HEP stands** — 74 scored science benchmarks: HEP brings the most, and few clear the bar

Part IV **Benchmarking well** — the question, baselines, metrics, splits and leakage, error bars, reading a result, score it yourself

A benchmark is a *standardized* version of a scientific application — in five elements

1 · Problem spec

- the task: input data, expected output
- system constraints: power, latency, hardware, as fixed bounds

e.g. jet features in → jet class out, within an FPGA latency and resource budget

2 · Dataset

- FAIR: findable, accessible, interoperable, reusable
- bounded: no augmentation unless specified
- one canonical train / val / test split

e.g. a versioned release with a DOI and fixed splits

3 · Metrics

- quantifiable, reproducible, computed on the held-out set
- can be multi-dimensional → a Pareto front

e.g. accuracy, AUC, latency, resource use

4 · Reference solution

- meets the spec and constraints
- reports every metric
- the baseline everyone compares to

e.g. a published, quantized MLP with its training code

5 · Docs & protocol

- reproduce the reference in a versioned, defined environment
- hardware references need a bill of materials

e.g. a pinned container and a one-line run command

HEP examples in the dashed rows are ours, in the spirit of the ontology's Jet Classification benchmark

TERMS **HEP** high energy physics **FPGA** field-programmable gate array, a reconfigurable chip used in trigger and readout electronics **FAIR** findable, accessible, interoperable, reusable **AUC** area under the ROC curve, a one-number summary of a classifier **MLP** multilayer perceptron, a basic fully connected neural network **DOI** digital object identifier, a permanent citable link to a dataset or paper **Pareto front** the set of best trade-offs: no metric improves without another getting worse **container** a packaged software environment with every dependency pinned

Constraints are *bounds*. Metrics are what you *compare*.

- **Constraint**: a fixed upper or lower limit, not part of the score

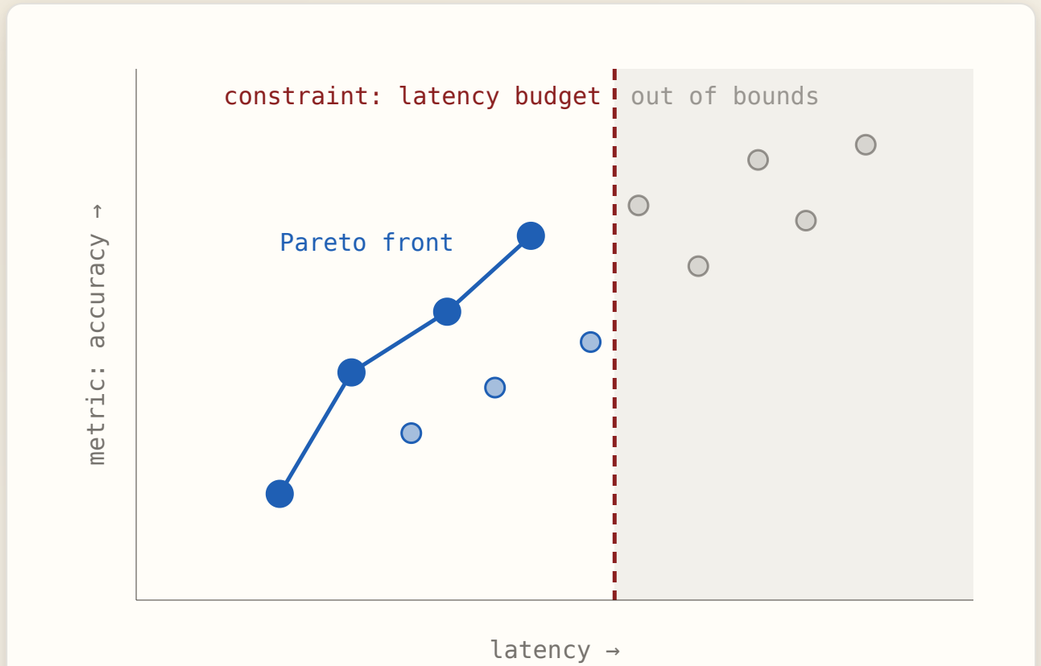
  power, latency, throughput, target hardware, technology node

- **Metric**: the measurable basis for comparison

  MSE on held-out data, conserved-quantity error, CPU/GPU time, memory; robustness when noise is added

- **One task, several benchmarks**

  same dataset: one benchmark fixes a minimum accuracy and tests throughput, another asks only for the best accuracy



illustrative — think of the Level-1 (L1) trigger: a fixed latency budget, then the best accuracy within it

TERMS **L1 trigger** Level-1 trigger, the hardware first stage that picks which collisions to keep, within microseconds **MSE** mean squared error **technology node** the chip manufacturing process generation

Six categories, *0–5* points each — one point per criterion met

Software environment

- baseline code available, complete, documented
- runs **without modification**
- containerized, or full setup steps

Problem specification

- system constraints given
- task is clear
- data format, inputs and outputs specified

Dataset

- one point per FAIR principle (4)
- +1 for well-defined train / test (/ val) splits

Performance metrics

- definition: 3 fully defined · 2 named, details missing · 1 vague · 0 none
- quality: 2 captures performance fully · 1 partially · 0 not

Reference solution

- public and documented
- hardware / software requirements listed
- every metric evaluated; model, hyperparameters and training code open

Documentation

- task, background, motivation explained
- evaluation criteria explained
- an academic paper exists

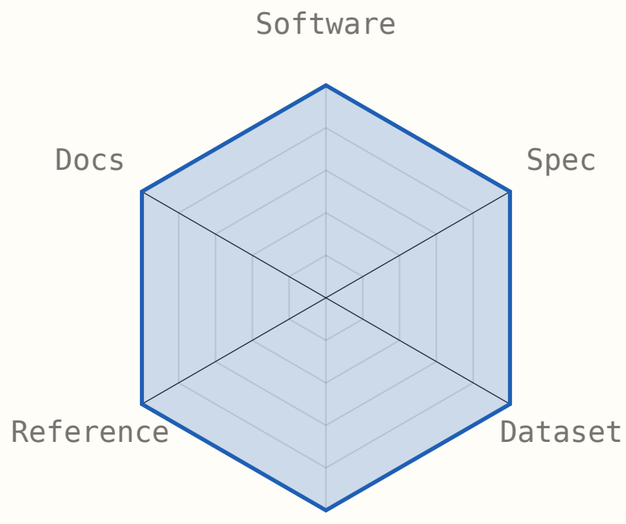
TERMS **hyperparameters** settings chosen before training, such as layer sizes or learning rate

THE BAR

Average ≥ *4.5 / 5* across all six

earns the **MLCommons Science Benchmark Endorsement**: a particularly high-quality benchmark within the wider ontology

- New benchmarks enter through an **open submission workflow**
  reviewed by the MLCommons Science Working Group
- The ontology is not static
  new domains, AI/ML motifs and computing motifs are added as workloads evolve



a 5.00 benchmark fills the hexagon

Part III — *where HEP stands*

Part I **Why benchmark** — what a benchmark is for, and why so few “benchmarks” can be rerun

Part II **What makes a good benchmark** — five elements, a rubric to score them, and the bar for “high quality”

Part III **Where HEP stands** — 74 scored science benchmarks: HEP brings the most, and few clear the bar

Part IV **Benchmarking well** — the question, baselines, metrics, splits and leakage, error bars, reading a result, score it yourself

74 tasks, *seven* domains, *eight* motifs

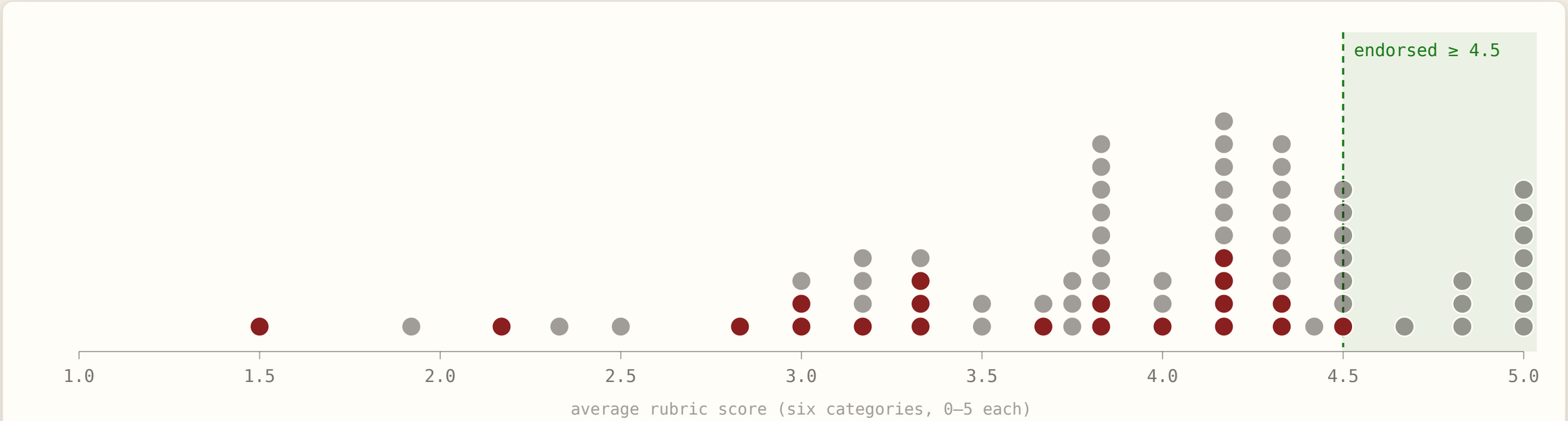
	Biology & Medicine	Chemistry	Climate & Earth	Comp. Science & AI	**High Energy Physics**	Materials Science	Mathematics
Anomaly Detection	1	0	2	0	4	0	0
Classification	6	0	5	2	8	3	0
Generative	0	2	0	1	2	1	0
Multimodal Reasoning	0	0	1	2	0	0	0
Reasoning & Generalization	5	3	3	2	3	1	4
Regression	1	5	4	1	1	5	2
Reinforcement Learning / Control	0	0	0	0	1	0	0
Sequence Prediction / Forecasting	3	0	2	1	1	0	0
entries	16	10	17	9	**20**	10	6

hover a cell for its benchmarks · a task can list several domains but only one AI/ML motif

- **Classification** and **reasoning** dominate
- **HEP contributes the most entries: 20 of 88**
  8 classification · 4 anomaly detection · 3 reasoning · 2 generative · regression, RL, forecasting
- Thin spots are opportunities
  multimodal reasoning, RL / control and forecasting barely appear in HEP or anywhere else

TERMS **AI/ML motif** the type of machine-learning task, e.g. classification or regression **entry** a task counted once for each domain it lists

HEP brings the most tasks, and *one* endorsement



● HEP task ● other domains

18 of 74 tasks endorsed · HEP mean **3.54** vs 3.93 overall · the one HEP endorsement: **CaloChallenge 2022**

every task in Table II of arXiv:2511.05614 · hover a dot for its name and score

The HEP entries — benchmarks *you* already know

benchmark	motif	score
CaloChallenge 2022 LHC [68]	Generative	4.50
GeSS - Track Pileup [73]	Classification	4.33
GeSS - Track Signal [73]	Classification	4.33
Jet Classification LHC [11]	Classification	4.17
Irregular Sensor Data Compression [14]	Generative	4.17
MLPerf HPC - Cosmoflow [36]	Regression	4.17
HDR ML Anomaly Challenge - Gravitational Waves [43]	Anomaly Detection	4.17
The Well [47] · multi-domain	Sequence Prediction / Forecasting	4.00
GPQA Diamond [78] · multi-domain	Reasoning & Generalization	3.83
Neural Architecture Codesign for Fast Physics Applications [20]	Classification	3.83

benchmark	motif	score
GPQA (graduate-level Google-proof Q&A) [80] · multi-domain	Reasoning & Generalization	3.67
CURIE (scientific long-context understanding and reasoning) [82] · multi-domain	Reasoning & Generalization	3.33
Smart Pixels for LHC LHC [19]	Classification	3.33
LHC New Physics Dataset LHC [83]	Anomaly Detection	3.33
Ultrafast jet classification at the HL-LHC LHC [17]	Classification	3.17
Beam Control [18,87]	Reinforcement Learning / Control	3.00
Intelligent experiments through real-time AI [44]	Classification	3.00
DUNE [89]	Anomaly Detection	2.83
Quench detection [45]	Anomaly Detection	2.17
In-Situ High-Speed Computer Vision [90]	Classification	1.50

LHC = built around LHC detectors or data · names from the ontology's source data (long ones shortened) · multi-domain entries (The Well, GPQA, CURIE) include HEP among their domains

TERMS **LHC / HL-LHC** Large Hadron Collider / its High-Luminosity upgrade **DUNE** Deep Underground Neutrino Experiment **HDR** the NSF Harnessing the Data Revolution program

Part IV — *benchmarking well*

Part I — **Why benchmark** — what a benchmark is for, and why so few “benchmarks” can be rerun

Part II — **What makes a good benchmark** — five elements, a rubric to score them, and the bar for “high quality”

Part III — **Where HEP stands** — 74 scored science benchmarks: HEP brings the most, and few clear the bar

Part IV — **Benchmarking well** — the question, baselines, metrics, splits and leakage, error bars, reading a result, score it yourself

Write the benchmark *before* the model

Decision what will this result change? → **Task** inputs, outputs → **Constraints** the bounds of the real deployment → **Metric** what "better" means → **Baseline** what you must beat

a CMS-flavoured example

- **Decision:** should a new tagger replace the current Level-1 algorithm?
- **Task:** trigger objects in → keep / reject out
- **Constraints:** the latency and FPGA resources the trigger board actually has
- **Metric:** signal efficiency at the fixed trigger rate
- **Baseline:** the algorithm running today, on the same events

- Fix all five **before** training anything
  choices made after seeing results drift toward whatever your model happens to do well
- Compare against the **traditional workflow**, not only other ML
  that comparison is the AI advantage, and the one decision makers ask about
- Several decisions → several benchmarks on one dataset

TERMS **tagger** a classifier that labels jets or other physics objects **trigger rate** how many collisions per second the trigger keeps
signal efficiency / rejection fraction of signal kept / 1 ÷ fraction of background kept

Three baselines *every* result needs

- **Trivial**: predict the mean, the majority class, random
  sets the floor; anything near it means the metric or the task is broken
- **Traditional**: the cut-based or physics-driven method in use today
  this is the comparison that justifies adopting a model
- **Simple ML**: a BDT or small MLP on the same inputs
  tells you whether the fancy architecture is what helps
- **Give every baseline the same tuning budget you gave your model**

A REAL LEADERBOARD: AXESS, FPGA RESOURCE AND LATENCY PREDICTION

Results

Task:

#	Participant	Date	ID	Test mean R2	Test mean SMAPE %	Test R2 BRAM	Test R2 DSP	Test R2 FF	Test R2 LUT	T
1	bhawks	2026-10-02 12:18	121	**0.809**	10.29	0.349	0.827	0.925	0.904	0
2	bhawks	2026-10-02 12:04	120	**-0.0**	113.96	-0.0	-0.0	-0.0	-0.0	-(

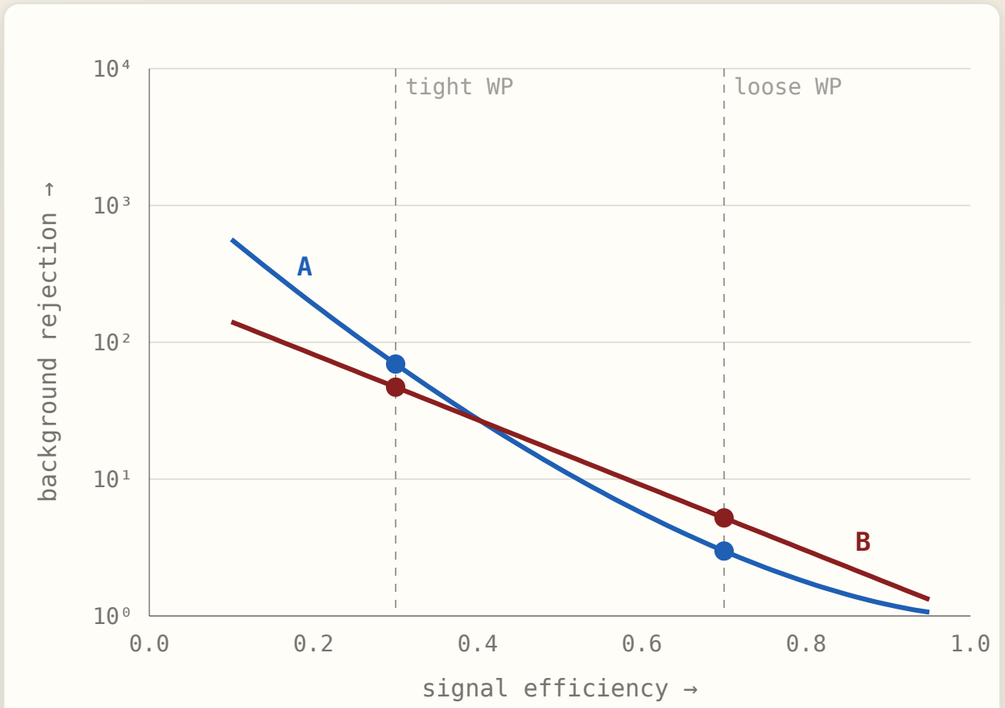
trained model: **R^2 0.81** · SMAPE 10.3%

predict-the-mean: **R^2 0.00** · SMAPE 114%

A big gap to the trivial entry is evidence the metric measures something real.

TERMS **BDT** boosted decision trees **R^2** coefficient of determination: 1 is perfect, 0 is no better than predicting the mean **SMAPE** symmetric mean absolute percentage error
AXESS a benchmark that predicts FPGA resources and latency for neural networks

Pick metrics that match the *physics*



illustrative curves · rejection = 1 / background efficiency

- One summary number can hide the **working point** you care about
  A wins at a tight working point, B at a loose one; an analysis lives at one of them
- **Fully define** it: formula, implementation, which split, which events
  "accuracy" is a name; the scoring function is the definition
- Several metrics → report all of them, or the Pareto front
  keep constraints such as latency as pass / fail bounds, not folded into a score
- **Once a metric becomes the target, people optimize the metric**
  so make sure that improving it really means improving the physics

TERMS **WP** working point, the cut on the classifier output that an analysis actually uses

The test set is *sacred* — and easy to contaminate

Same event, both sides

- several jets or candidates from one event, split across train and test
- reused MC samples or generator seeds

fix: split by event or sample, not by row

Truth sneaking in

- inputs derived from Monte Carlo (MC) simulation truth
- labels encoded in file order, sample names or weights

fix: list the forbidden fields in the spec

Tuning on test

- hyperparameter search or early stopping on the test split
- many submissions against one test set

fix: a validation split, and a held-out test touched once

Distribution shift

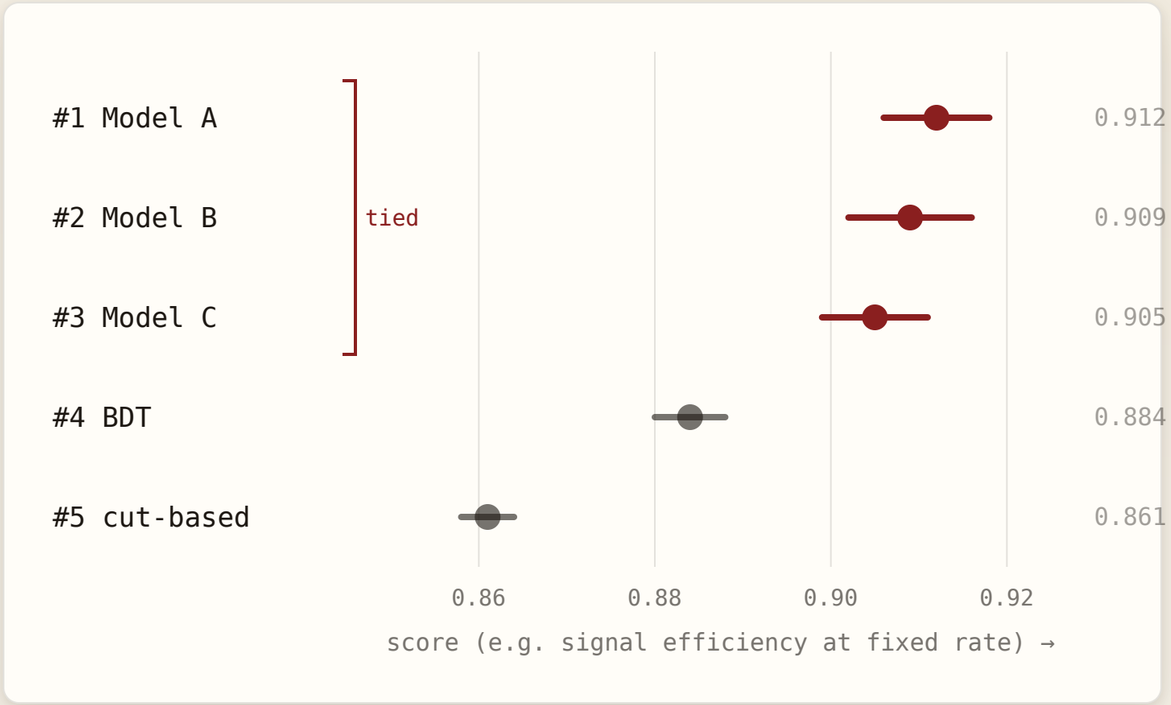
- train on simulation, deploy on data
- different pileup or run conditions

fix: state it, and test on the shifted set too

The ontology's answer: **one canonical, non-overlapping split**, curated once, and a **bounded** dataset with no augmentation unless the benchmark specifies it.

TERMS **MC** Monte Carlo simulation of collisions and the detector **pileup** extra proton-proton collisions in the same bunch crossing **seed** the random number that sets a model's initialisation or a generator's output **early stopping** ending training when a held-out score stops improving

Is #1 actually *better*?

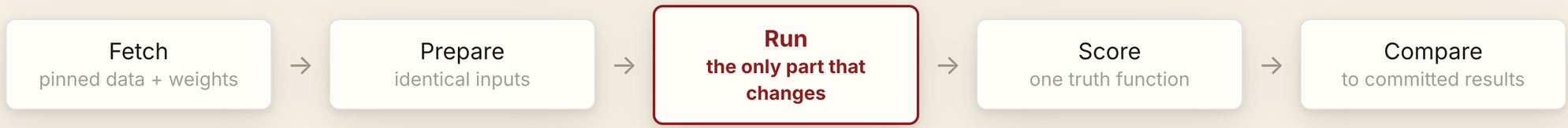


illustrative leaderboard · bars show the spread over training seeds and test-set bootstraps

- You'd never quote a cross-section without an uncertainty
  a leaderboard score is a measurement too
- **Seeds**: retrain with several random seeds; report mean and spread
- **Test set**: bootstrap it to see how much the score moves
- **Differences smaller than the spread are ties**
  here, the top three are statistically indistinguishable

TERMS **bootstrap** resampling the test set with replacement to see how much the score moves

Same steps, same scorer, *every* model



- The reference solution and every new model go through **exactly the same steps**

  different preprocessing or a private scoring script is how two papers end up "comparing" different things

- A new model plugs in as **one predictions file**, scored by the shared function

- Commit the reference numbers, so "did I reproduce it?" is a **diff**

  AXESS: a workstation CPU run and a GPU run on an HPC system agreed to 2.4×10^{-4}

- Reproducing on different hardware is itself a test of the benchmark

TERMS **truth function** the single scoring code that compares predictions with the true labels **CPU / GPU / HPC** central / graphics processing unit / high-performance computing

Lessons that *generalize*

Check before you trust

- Make sure you ran what you think you ran
  same model, same version, same code
- Make sure you're scoring the right answer
  we found published checkpoints trained on a different label than the one being scored
- Share the whole recipe, not just the model
  preprocessing, prompts and settings matter too
- Reproduce the published numbers
  if they don't match, find out why and write it down

Make it easy for others to trust

- Pin your versions
  software and data, so a rerun next year matches
- Test on a few real examples
  catches silent breakage in seconds
- Keep a log of what you checked
  what, the result, when, and on which machine
- Be upfront about limitations
  say where your benchmark falls short

Reading isn't verifying.

TERMS **checkpoint** a saved copy of a model's trained weights

Seven questions to ask of *any* benchmark result

1 What is the baseline?
trivial, traditional and simple-ML, tuned as hard as the winner

2 Is the metric the one I care about?
at my working point, under my constraints

3 Were the constraints met?
latency, resources or power as bounds, not footnotes

4 Was the test set ever seen?
by training, tuning, early stopping or repeated submissions

5 How big is the uncertainty?
seeds and test-set bootstrap; are the top entries tied?

6 Does it transfer?
simulation to data, other pileup, other run conditions

7 Can I rerun it?
public code, pinned environment, committed reference numbers

Most of these are the rubric's categories, asked as a reader.

Worked example — what a *5 / 5* benchmark looks like

AXESS: from a neural network and its hls4ml settings, predict FPGA resources and latency without running hours of synthesis

Problem spec

- NN config in → FPGA resources + latency out; target device fixed as a constraint

Dataset

- public, versioned, one fixed set of splits

Metrics

- R^2, SMAPE, RMSE with formulas written out, plus timings

Reference solution

- GNN and Transformer surrogates, compared against running synthesis

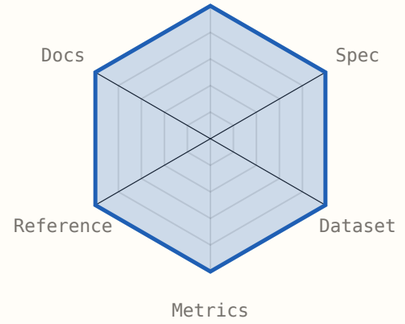
Docs & protocol

- pinned environments, scripted jobs, committed reference results, a peer-reviewed paper

RUBRIC SELF-SCORE

5.00 / 5

Software



MLCommons endorsement bar ≥ 4.5 ✓

TERMS **hls4ml** an open-source tool that turns neural networks into FPGA firmware **synthesis** compiling a hardware design for an FPGA; can take hours per model **GNN / Transformer** graph neural network / attention-based neural network **RMSE** root mean squared error

Score it yourself — *would it be endorsed?*

Software environment 1

- [x] Code to reproduce the baseline reference solution is available
- [ ] The provided code is complete
- [ ] The code itself is well documented
- [ ] The code runs without modification
- [ ] Containerized, or environment details and setup instructions provided

Problem specification 4

- [ ] System constraints (power, latency, throughput) are provided
- [x] The benchmark task is clear
- [x] The dataset format is clearly specified
- [x] The task inputs are clearly specified
- [x] The task outputs are clearly specified

Dataset 4

- [x] Findable
- [x] Accessible
- [x] Interoperable
- [ ] Reusable
- [x] Well-defined train / test (/ validation) splits

Performance metrics 3

metric definitions

- ○ 3 — fully defined
- ◉ 2 — named, details missing
- ○ 1 — mentioned, unclear what is tracked
- ○ 0 — none

metric quality

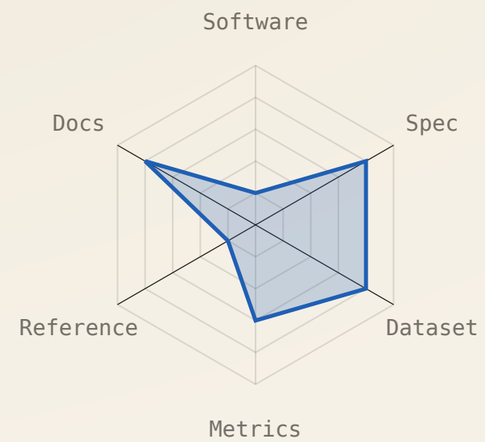
- ○ 2 — fully captures performance
- ◉ 1 — partially
- ○ 0 — does not

Reference solution 1

- [x] A reference solution is publicly available
- [ ] It is well documented
- [ ] All hardware / software requirements are listed
- [ ] Every metric of the benchmark is evaluated on it
- [ ] The baseline model is open for study (architecture, hyperparameters, training code)

Documentation 4

- [x] The task is explained and well documented
- [x] Task and benchmark background is clearly explained
- [x] The motivation is clearly explained
- [ ] The evaluation criteria are clearly explained
- [x] An academic paper about the benchmark exists



2.83 / 5

below the 4.5 bar (1.67 short)

“dataset + paper” AXESS

clear

pick a benchmark you use. “dataset + paper” is an illustrative preset, not a real rating

Where to *go next*

what	for	where
The ontology paper	the definition, the rubric, the motifs, Table II	arXiv:2511.05614
Ontology website	browse, filter and sort every benchmark; table view exports CSV / JSON	mlcommons-science.github.io/benchmark
Submit a benchmark	YAML definitions + CONTRIBUTING.md, reviewed by the MLCommons Science WG	github.com/mlcommons-science/benchmark
HEP starting points	ontology entries: Jet Classification · CaloChallenge 2022 · Ultrafast jet classification at the HL-LHC · LHC New Physics Dataset	2207.07958 · 2410.21611 · 2402.01876 · 2107.02157
Benchmark Builder	optional: an AI-agent skill that structures a benchmark and self-scores it against the rubric	github.com/ben-hawks/benchmark-builder

Homework: take one model comparison from your analysis or group and ask it the seven questions.

Thank you

Start from the decision · beat honest baselines · measure at the working point · protect the test set · put error bars on it

arXiv:2511.05614 · mlcommons-science.github.io/benchmark

Design adapted from Andrzej Novak's CMS masterclass talk (jfc-mit/jfc-talk) · full credits and a glossary in the backup slides

One benchmark, *three* kinds of reader

a specific benchmark

- “I need a classification task on calorimeter images to test my method”
- search the metadata of every benchmark

who: **you**, with a model to evaluate

a benchmark category

- “all HEP benchmarks”, or “all anomaly detection in HEP”
- filter and sort by domain and AI/ML motif

who: **domain scientists** surveying a field

similar compute patterns

- “workloads that behave like mine on a GPU, in any domain”
- cluster benchmarks by measured behaviour, not by label

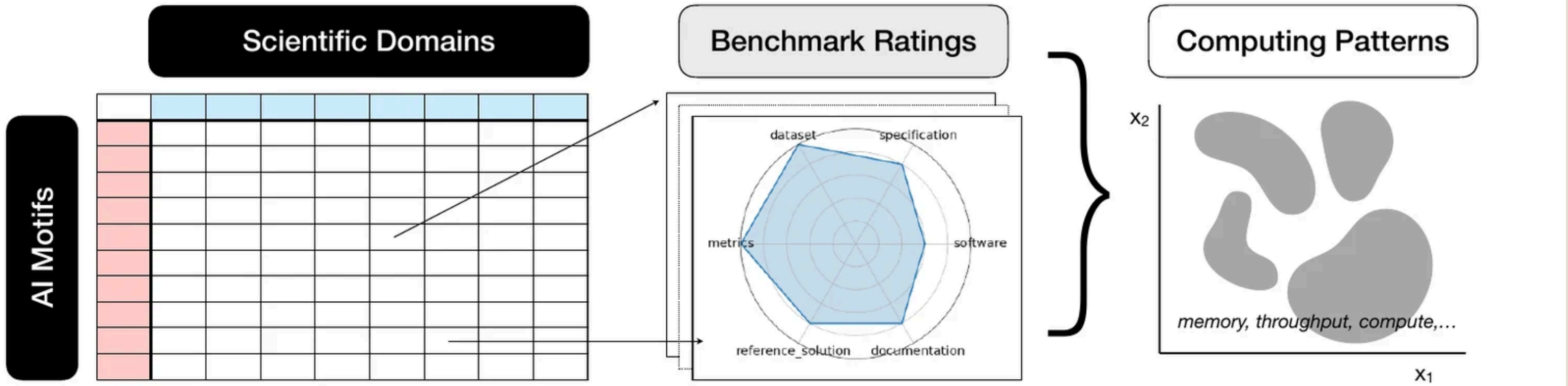
who: **hardware vendors** and **systems researchers**

Different readers value different things, so the ontology has to answer all three.

Built on *prior* benchmark efforts

work	focus	scope	limitation the ontology addresses
XAI-Bench	explainable AI	synthetic data with ground-truth feature importance	interpretability metrics only, not end-to-end scientific ML
FastML Science Benchmarks	edge ML for physics	real-time, low-latency: HEP jet tagging, sensor data compression, accelerator control	first to stress hardware/software co-design and real-time constraints, but domain specific
PDEBench	PDE surrogates	11 PDEs, forward and inverse problems; FNO, U-Net, PINN baselines	PDE simulation only
SciMLBench	general framework	scientific, application and system-level benchmarks	a blueprint more than an implemented suite
This work	end-to-end scientific ML	community-driven, many domains, curated for quality	one ontology under MLCommons governance: reproducibility, FAIR data, open submissions

Taxonomy → *ratings* → computing patterns



Classify every task by scientific domain and AI/ML motif

Qualify each benchmark with the six-category rubric

Characterize workloads by measured behaviour: memory, throughput, compute

Fig. 1 of arXiv:2511.05614

Computing motifs — what *bounds* the workload

Latency bound

- a fixed latency constraint, or "produce an output as fast as possible"

HEP: Level-1 trigger inference on FPGAs

Memory bound

- tasks that need large amounts of data at once

HEP: graph-based tracking on a full high-pileup event

Throughput bound

- consume and produce data at very high rates, often limited by bandwidth

HEP: on-detector data reduction, 40 MHz scouting

Utilization bound

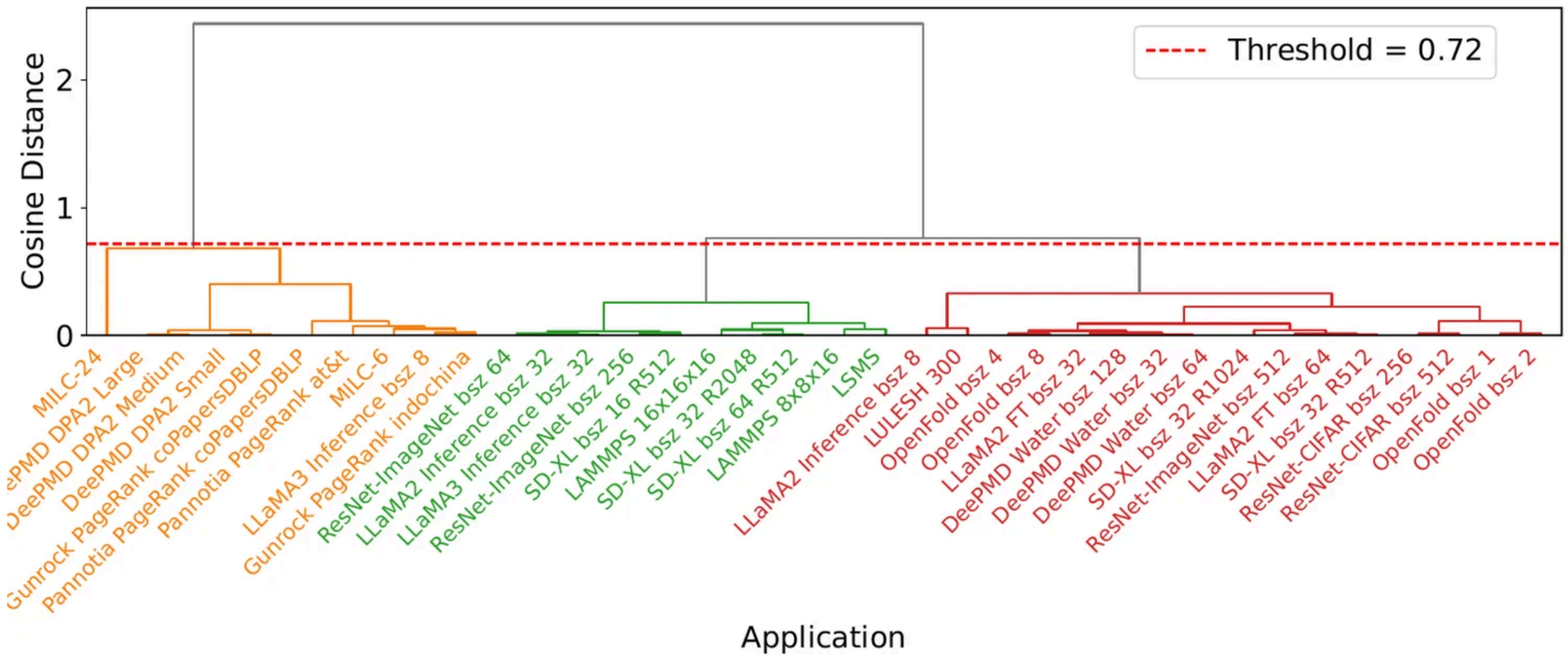
- a fixed compute limit, or an inherently expensive task

HEP: detector simulation and its fast surrogates

A motif is a property of an **implementation** more than of the benchmark itself, and one workload can be bound by several.

definitions from §IV-C of the paper; the HEP examples are ours

Find similar workloads by *measuring* them



- Profile each benchmark solution's power with `nvidia-smi` / NVML

  graph analytics, HPC, HPC+ML and ML GPU workloads; different inputs counted separately

- Hierarchical clustering on **cosine distance**; cut at 0.72 → **K = 3**

  low-power · high-power · mixed. A user's priorities pick the axes, so the clusters adapt to what they care about

Fig. 3 of arXiv:2511.05614

Backup — a benchmark card *is* the README

Header · motifs

- scientific motif
- AI/ML motif
- computing motif (optional)

1 · Problem spec

- task
- inputs & outputs
- system constraints

2 · Dataset

- summary & provenance
- splits, schema, access
- FAIR checklist, bounded-ness

3 · Metrics

- primary + secondary metrics
- scoring definition

4 · Reference solution

- summary & code location
- architecture / method
- results on every metric
- hardware / software requirements

5 · Docs & protocol

- reproduction steps
- environment (container)
- motivation, background, citation

Submission guidelines

- what to submit, and how

Known limitations

- stated openly, not hidden

Backup — a repository layout that *reproduces*

path	what it's for
README.md	the benchmark card
data/SCHEMA.md	inputs, outputs, forbidden fields
src/<pkg>/score.py	the one truth + metrics function
weights/MANIFEST.json	pinned weights / model revisions
reference_results/ + tests/	committed numbers + golden-output tests
jobs/	batch scripts with small per-machine profiles

Glossary — every term, *in one place*

AI advantage a measured gain over the traditional workflow, on the benchmark's own metrics

AI/ML motif the type of machine-learning task, e.g. classification or regression

AUC area under the ROC curve, a one-number summary of a classifier

AXESS a benchmark that predicts FPGA resources and latency for neural networks

BDT boosted decision trees

bootstrap resampling the test set with replacement to see how much the score moves

checkpoint a saved copy of a model's trained weights

container a packaged software environment with every dependency pinned

CPU / GPU / HPC central / graphics processing unit / high-performance computing

DOI digital object identifier, a permanent citable link to a dataset or paper

DUNE Deep Underground Neutrino Experiment

early stopping ending training when a held-out score stops improving

entry a task counted once for each domain it lists

FAIR findable, accessible, interoperable, reusable

FPGA field-programmable gate array, a reconfigurable chip used in trigger and readout electronics

GNN / Transformer graph neural network / attention-based neural network

HDR the NSF Harnessing the Data Revolution program

HEP high energy physics

hls4ml an open-source tool that turns neural networks into FPGA firmware

hyperparameters settings chosen before training, such as layer sizes or learning rate

L1 trigger Level-1 trigger, the hardware first stage that picks which collisions to keep, within microseconds

LHC / HL-LHC Large Hadron Collider / its High-Luminosity upgrade

MC Monte Carlo simulation of collisions and the detector

MLCommons the open engineering consortium behind the MLPerf benchmarks

MLP multilayer perceptron, a basic fully connected neural network

MSE mean squared error

ontology a structured catalogue: shared definitions, categories and ratings

Pareto front the set of best trade-offs: no metric improves without another getting worse

pileup extra proton-proton collisions in the same bunch crossing

R^2 coefficient of determination: 1 is perfect, 0 is no better than predicting the mean

RMSE root mean squared error

seed the random number that sets a model's initialisation or a generator's output

signal efficiency / rejection fraction of signal kept / 1 ÷ fraction of background kept

SMAPE symmetric mean absolute percentage error

synthesis compiling a hardware design for an FPGA; can take hours per model

tagger a classifier that labels jets or other physics objects

technology node the chip manufacturing process generation

trigger rate how many collisions per second the trigger keeps

truth function the single scoring code that compares predictions with the true labels

WP working point, the cut on the classifier output that an analysis actually uses

Credits and *sources*

- **Presentation design and engine** adapted from Andrzej Novak's CMS internal masterclass talk on the JFC paper (22 Sep 2026)

  github.com/jfc-mit/jfc-talk · itself styled after the hcc-kappac Florence talk and danielmurnane.com · paper theme, mono crumbs, the part outline and scene/step flow follow that deck

- **Benchmark definition, rubric, Table II data, Figs. 1 and 3**

  B. Hawks, G. von Laszewski, M. D. Sinclair, M. Colombo, S. Venkataraman, R. Jain, Y. Jiang, N. Tran, G. Fox, “An MLCommons Scientific Benchmarks Ontology”, arXiv:2511.05614

- **Part IV examples** (AXESS, baselines, one scorer, lessons) adapted from B. Hawks, “Benchmarking-driven model development”, tutorial session, 15 Oct 2026

- **HEP benchmark names** in Part III are taken from the ontology’s source data, mlcommons-science/benchmark

- **Typeface** EB Garamond (SIL Open Font License), via Google Fonts